

Clase 241 — Secretos en el código y pre-commit hooks

Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: *Securing DevOps* (Julien Vehent) y OWASP Secrets Management Cheat Sheet 🛡️ Duración estimada: **90 min** · Nivel: **Intermedio**

🎯 Objetivo

Aprender a prevenir, detectar y remediar la filtración de secretos (API keys, tokens, contraseñas, claves privadas) en repositorios de código. Un secreto commiteado —aunque se borre después— queda en el historial de Git y debe considerarse comprometido. Implementaremos una defensa en profundidad: detección en el historial, hooks de pre-commit que bloquean antes de commitear, y escaneo en CI, usando **gitleaks** y el framework **pre-commit**.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** por qué borrar un secreto de HEAD no lo elimina del historial ni del riesgo.
2. **Detectar** secretos en el historial completo de un repositorio con gitleaks.
3. **Configurar** un hook de pre-commit que bloquee commits con secretos.
4. **Integrar** el escaneo de secretos en CI como red de seguridad.
5. **Remediar** correctamente: rotar el secreto y purgar el historial.

📖 Temas

#	Tema	Por qué importa
1	Anatomía de una fuga de secreto	Entender el ciclo de vida del riesgo
2	Detección por entropía y patrones	Cómo encuentran los escáneres los secretos
3	gitleaks: historial y pre-commit	Herramienta principal
4	El framework pre-commit	Orquestrar hooks locales reproducibles
5	Escaneo en CI	Red de seguridad si el hook se salta
6	Remediación: rotar y purgar	Detectar no basta; hay que rotar
7	Gestión de secretos correcta	Vaults en vez de secretos en código

📖 Definiciones y características

- **Secreto:** credencial que da acceso (API key, token, password, clave privada). *Característica:* si toca un repo, asume compromiso y rota.

- **Detección por entropía:** identificar strings con alta aleatoriedad típica de claves. *Característica:* complementa a las reglas de patrón (regex).
- **Pre-commit hook:** script que corre antes de crear el commit. *Característica:* bloquea el secreto antes de que entre al historial; local y evitable, por eso se combina con CI.
- **Historial de Git:** todos los commits pasados. *Característica:* un secreto borrado sigue accesible vía `git log / git show` de commits antiguos.
- **Rotación:** invalidar el secreto expuesto y emitir uno nuevo. *Característica:* es la única remediación real; purgar el historial no revierte la exposición.
- **Secrets manager / vault:** sistema que almacena y sirve secretos fuera del código. *Característica:* la solución de raíz (Vault, AWS Secrets Manager, etc.).

Herramientas y preparación

- **gitleaks** — detección de secretos en historial y en pre-commit.
- **detect-secrets** (Yelp) — alternativa con baseline de allowlist.
- **pre-commit** (framework) — orquesta hooks locales de forma reproducible.
- **git-filter-repo** / BFG Repo-Cleaner — para purgar secretos del historial.

Instalación:

```
# gitleaks
brew install gitleaks      # o descarga el binario de releases

# framework pre-commit
pip install pre-commit
```

Nota ética: nunca subas secretos reales a repos públicos para "probar". Usa secretos falsos de ejemplo. Si detectas un secreto de tu organización, trátalo como incidente: rota primero.

Laboratorio guiado

1. **Escanea el historial completo** de un repositorio de práctica:

```
gitleaks detect --source . --report-format json --report-path gitleaks.json -v
```

Revisa hallazgos: cada uno lista commit, archivo, línea y regla. 2. **Instala el hook de pre-commit.** Crea `.pre-commit-config.yaml`:

```
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.0
  hooks:
  - id: gitleaks
```

```
pre-commit install
```

3. **Prueba el bloqueo.** Añade una línea con un token falso de ejemplo (`AKIAIOSFODNN7EXAMPLE`) e intenta commitear: el hook debe abortar el commit.
4. **Gestiona falsos positivos.** Crea un `.gitleaks.toml` con una allowlist para valores de ejemplo/tests, con justificación.
5. **Escaneo en CI.** Añade un job que corra `gitleaks detect` en cada push; es la red de seguridad si alguien no tiene el hook local. Usa la action oficial `gitleaks/gitleaks-action`.
6. **Simula remediación.** Ante un secreto detectado: (a) rota el secreto en su proveedor (paso conceptual), (b) purga el historial con `git filter-repo --path archivo --invert-paths` o BFG, (c) fuerza push coordinado con el equipo. Documenta que la rotación es lo primero.
7. **Mueve el secreto a un vault.** Reemplaza el secreto hardcodedo por una variable de entorno inyectada desde un secrets manager.

Ejercicios

1. Escanea el historial de un repo y clasifica los hallazgos por severidad.
2. Configura pre-commit con gitleaks y demuestra que bloquea un commit.
3. Crea una allowlist para valores de ejemplo con justificación.
4. Añade el escaneo de secretos en CI y haz que falle el build.
5. Documenta el procedimiento de remediación paso a paso (rotar primero).
6. Refactoriza una app para leer un secreto desde variable de entorno/vault.

Reto verificable

Implementa defensa en profundidad contra secretos en un repositorio.

Criterio de aceptación: (a) hay un hook de pre-commit con gitleaks que bloquea secretos localmente; (b) CI escanea el historial en cada push y falla ante un hallazgo; (c) existe una allowlist justificada para falsos positivos; y (d) se entrega un runbook de remediación que pone la rotación del secreto como primer paso, antes de purgar el historial.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Borré el secreto en un nuevo commit, ya está"	Sigue en el historial. Rota el secreto y purga con filter-repo/BFG.
El hook no se ejecuta	Falta <code>pre-commit install</code> o el archivo de config. Verifícalo con <code>pre-commit run --all-files</code> .
Muchos falsos positivos en tests	Valores de ejemplo detectados como secretos. Añade allowlist en <code>.gitleaks.toml</code> .
Solo se escanea HEAD, no el historial	Usaste <code>--no-git</code> o un modo limitado. Usa <code>gitleaks detect</code> sobre todo el repo.
Se purgó el historial pero el secreto sigue funcionando	No se rotó. Purgar no invalida la credencial; rotar sí.

? Preguntas frecuentes

? **Un secreto estuvo 5 minutos en un repo privado y lo borré, ¿hay riesgo?** Sí. Asume compromiso y rota. Repos privados se clonan, se cachean y pueden volverse públicos por error; los bots escanean GitHub en segundos.

? **¿Los hooks de pre-commit son suficientes?** No por sí solos: son locales y evitables (`--no-verify`). Combínalos siempre con escaneo en CI del lado servidor.

? **¿Qué hago si el secreto está en cientos de commits?** Rota primero. Luego purga con `git-filter-repo` o BFG y coordina el force-push con todo el equipo (reescribe el historial compartido).

? **¿Cómo evito el problema de raíz?** No pongas secretos en el código. Usa un secrets manager/vault e inyéctalos en runtime como variables de entorno o montajes.

🔗 Referencias

- gitleaks — <https://github.com/gitleaks/gitleaks>
- OWASP Secrets Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- pre-commit framework — <https://pre-commit.com/>
- git-filter-repo — <https://github.com/newren/git-filter-repo>
- detect-secrets — <https://github.com/Yelp/detect-secrets>

➡ Siguiente clase

Clase 242 - Seguridad en pipelines CI/CD